

Data Structures and Algorithms

Recurrence Relation

Indian Institute of Information Technology Sri City, India

What is recursion?

- A program is recursive when it contains a call to itself.
 - Recursion can substitute iteration in program design
- Generally, recursive solutions are
 - Simpler than iterative solutions.
 - Slightly less efficient than the iterative ones (unless the recursive calls are optimized)
 - There are natural recursive solutions that can be extremely inefficient

Some Simple Examples

- Finding the factorial of a number

Code:

```
int factorial(int n) { //recursive soln.  
    if(n == 0) return 1;  
    else return n * factorial(n - 1);  
}
```

- Euclid's Algorithm for computing GCD

```
int gcd(int a, int b) {  
    if ( b == 0 ) return a;  
    return gcd(b, a%b);  
}
```

Recurrence Relations

- A recursive problem is one which can be expressed as a combination of its sub-problems (problems of smaller size)
- Eg: $fact(n) = n * fact(n-1)$
where $fact(n-1)$ is a sub-problem of $fact(n)$
- The complexity of recursive solutions are also usually represented in terms of smaller sizes of itself using recurrence relations

Example of Recurrence Relations

Consider the function:

```
void f(int n) {  
    if(n > 0) {  
        DoSomething(n);    // O(n)  
        f(n/2);  
    }  
}
```

The recurrence relation is expressed as:

$$T(n) = T(n/2) + O(n)$$

where the $T(n/2)$ comes because at each stage, the problem size is reduced by half, and the term $O(n)$ is used to denote work equivalent to $O(n)$ being done in the function

Example of Recurrence Relations

Consider the function:

```
void f(int n) {  
    if(n > 0) {  
        DoSomething(n);    // O(n)  
        f(n/2);    f(n/2);  
    }  
}
```

The recurrence relation is expressed as:

$$T(n) = 2T(n/2) + O(n)$$

where the $T(n/2)$ comes because at each stage, the problem size is reduced by half, and the term $O(n)$ is used to denote work equivalent to $O(n)$ being done in the function

Solving Recurrence Relations

i.e. for obtaining asymptotic “ Θ ” or “ O ” bounds on the solution:

- Substitution Method
- Recursion Tree Method
- Master Theorem

Substitution Method

- The basic idea is to guess a solution and then prove it using induction
- *Eg:* $T(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2T(n/2) + O(n), & \text{for all } n > 1 \end{cases}$
- Assume that the solution is $T(n) = n \log n + n$
 - Prove by induction

Substitution Method

1) $T(n) = n \log n + n$

2) Proof by Induction

Basis: if $n = 1$, $\Rightarrow 1 \log 1 = 1 \rightarrow T(n) = 1$, true for $n = 1$

Induction Step:

$$T(n) = 2T(n/2) + n = 2((n/2) * \log(n/2) + n/2) + n$$

$$= n \log(n/2) + n + n$$

$$= n(\log n - \log 2) + n + n = n \log n - n + n + n$$

$T(n) = n \log n + n$ is true for all $n > 1$